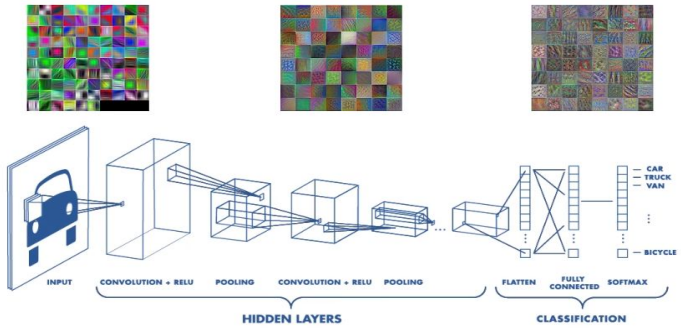
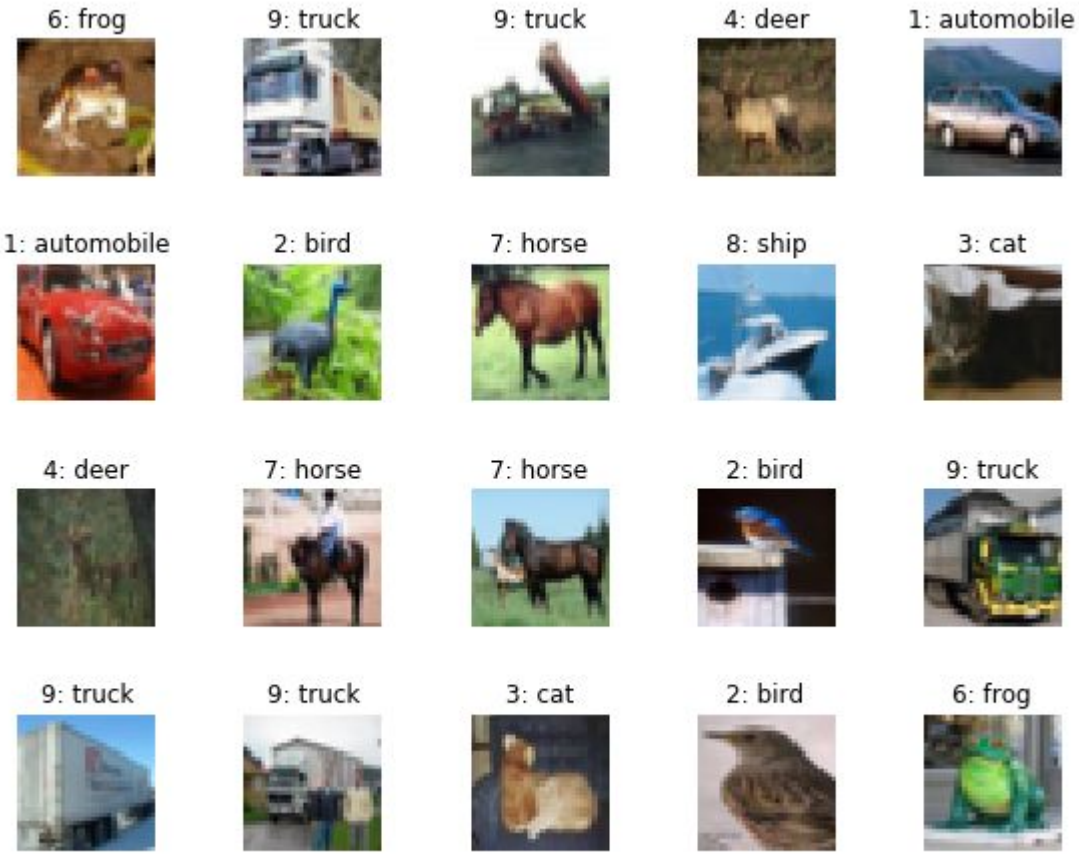
Redes neuronales convolucionales



CIFAR Dataset

Contiene:

- 60,000 imágenes (50k training)
- tamaño: 32×32 píxeles **(32,32,3)**
- imágenes a color (3 canales)
  - rojo, verde, azul
- 10 clases
- 6,000 imágenes por clase



Tipos de redes neuronales

- Redes neuronales artificiales básicas (ANN)
  - Predicción de precios y clasificación
- Redes neuronales convolucionales (CNN)
  - Clasificación de imágenes
- Redes neuronales recurrentes (RNN)
  - Texto y audio
- Transformer
  - GPT-4.0, deepseek-R1, etc

Redes neuronales convolucionales (CNN)

- Modelo de Deep Learning ideal para datos visuales tales como imágenes o videos, y algunas veces audios.
- Diseñadas para procesar datos con una estructura en forma de cuadricula como las imágenes.
- Permiten extraer características relevantes como bordes, texturas o formas.
  - Un grupo de pixeles puede representar parte de un borde o forma.
  - Las redes neuronales tradicionales tratan cada pixel como una característica independiente.

Redes neuronales convolucionales (CNN)

Una convolución es una operación matemática usada para detectar patrones en datos, especialmente imágenes.

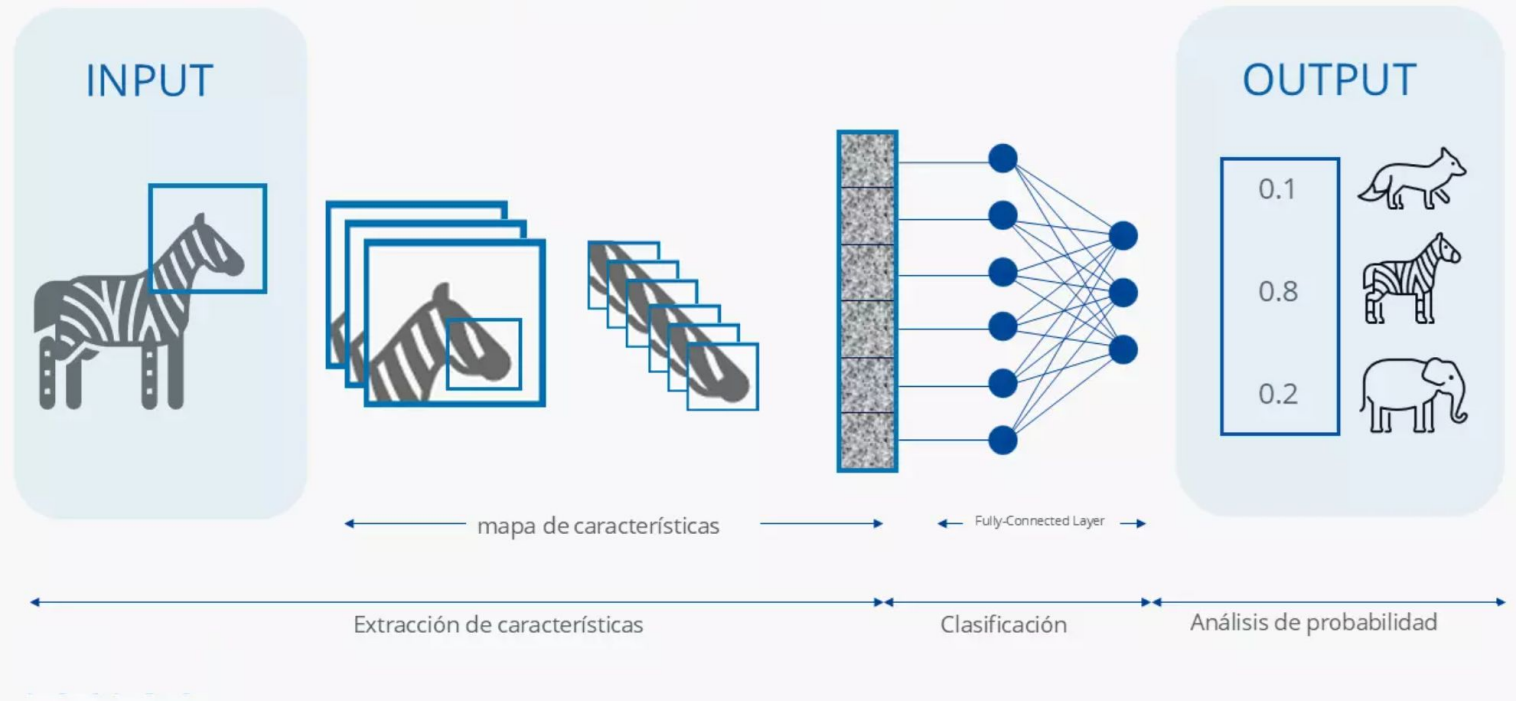
La idea simple es:

- Tomar una pequeña matriz llamada kernel o filtro.
- En cada posición se realizan **multiplicaciones y sumas**.
- El resultado **resalta** ciertas características.

Por ejemplo, un filtro puede detectar:

- bordes
- líneas
- texturas
- esquinas

Componentes



1. Capa convolucional
2. Capa de activación
3. Capa de agrupamiento (pooling)
4. Capa completamente conectada (fully connected)
5. Capa de salida

Posee dos fases: **Extracción de características y clasificación**

Componentes CNN

Capa convolucional

- Núcleo de la **CNN**
- Aplica filtros que generan un **mapa de características** para resaltar un tipo especifico de patrón visual como borde, textura, forma, etc)

Capa convolucional

```
model = models.Sequential([
    # Bloque 1
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    # Bloque 2
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Bloque 3
    layers.Conv2D(64, (3, 3), activation='relu'),
```

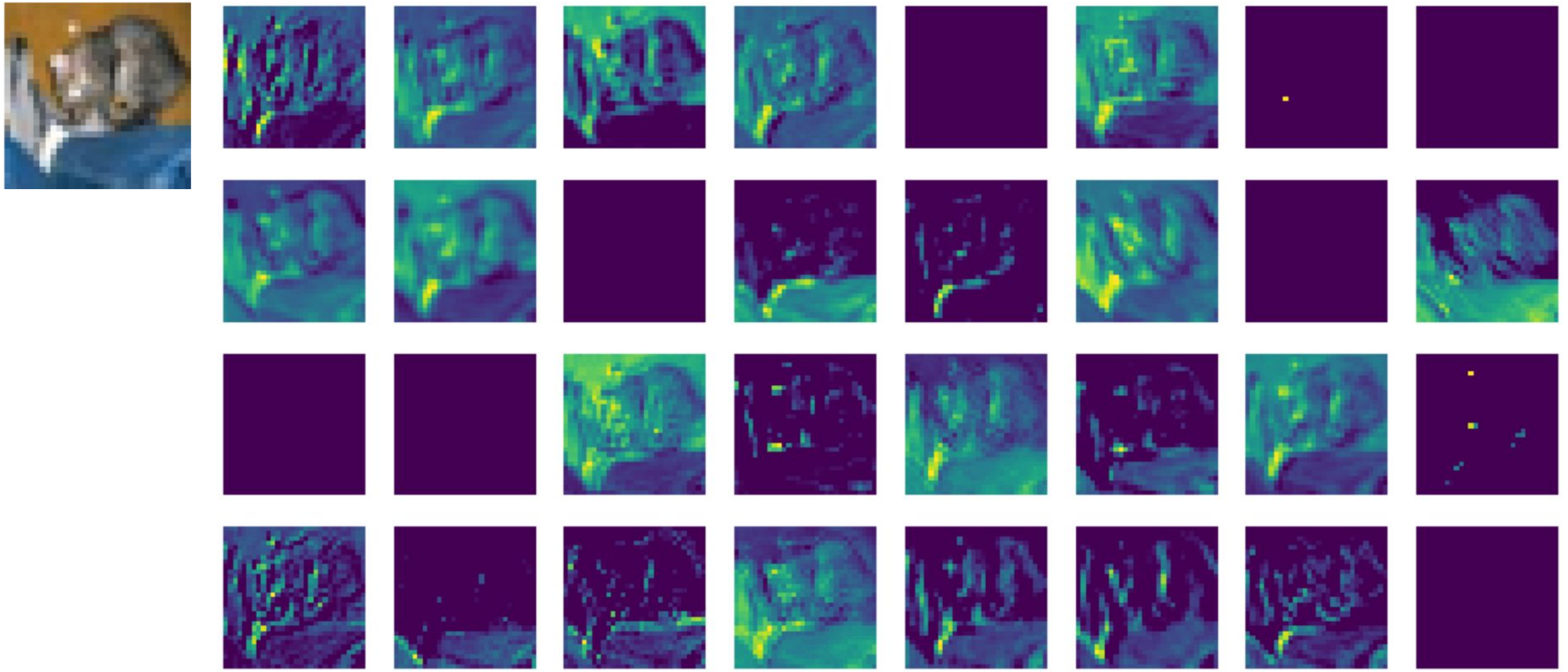
Conv2D(32) → Conv2D(64) → Conv2D(64)

- **32 filtros para captar caracteristicas basicas, generando 32 mapas de caracteristicas basicas.**
- **64 filtros para detectar combinaciones de rasgos (por ejemplo: formas y ojos)**
- **64 filtros para refinar y detectar patrones mas complejos**

Consideraciones:

- Más filtros = más capacidad de aprendizaje,
  - (pero) también más parámetros y más tiempo de entrenamiento.
- **Demasiados filtros** pueden provocar sobreajuste, especialmente si el dataset es pequeño.
- **Pocos filtros** pueden hacer que la red no capture suficiente información visual.
- En la práctica se eligen valores potencias de 2 (16, 32, 64, 128, …) por eficiencia en GPU.

Ejemplo de la salida del bloque uno, que crea 32 mapas de características



Como decidir cuantos filtros utilizar

Capa	Filtros típicos	Propósito	Observación
1ª capa	16–32	Detectar bordes, formas simples	Primer nivel de detalle
2ª capa	32–64	Detectar patrones más complejos (ojos, texturas)	Más abstracción
3ª capa	64–128	Detectar combinaciones de patrones (rostros, objetos)	Aumenta la profundidad
4ª o más	128–512	Detectar estructuras completas (perro, auto, avión)	Solo útil en redes grandes

Componentes CNN

Capa de agrupamiento (pooling)

- Reduce el tamaño de los mapas disminuyendo el numero de parámetros para evitar sobreajuste.

Ejemplo capa de agrupamiento (pooling)

Reducir el tamaño de los mapas

```python
import numpy as np
import tensorflow as tf

# Creamos una "imagen" de 1 canal (por ejemplo, un mapa de características)
# con valores fácilmente distinguibles
imagen = np.array([
    [1, 3, 2, 4],
    [5, 6, 7, 8],
    [3, 2, 1, 0],
    [4, 5, 6, 7]
], dtype=np.float32)

# Convertimos la imagen a formato tensor 4D (batch, alto, ancho, canales)
imagen_tensor = tf.reshape(imagen, (1, 4, 4, 1))

# Definimos la capa de max pooling 2x2
max_pool = tf.keras.layers.MaxPooling2D(pool_size=(2, 2))

# Aplicamos el max pooling
resultado = max_pool(imagen_tensor)

# Mostramos los resultados
print("Imagen original:\n", imagen)
print("\nDespués de aplicar MaxPooling2D((2, 2)):\n", tf.squeeze(resultado).numpy())
```

```
Imagen original:
 [[1. 3. 2. 4.]
 [5. 6. 7. 8.]
 [3. 2. 1. 0.]
 [4. 5. 6. 7.]]

Después de aplicar MaxPooling2D((2, 2)):
 [[6. 8.]
 [5. 7.]]
```

Tipos de pooling

- Max Pooling
  - Toma el valor máximo de cada ventana (2,2)
  - Conserva la característica mas intensa, hace que la red enfatice bordes, rasgos mas distintivos y patrones muy marcados.
- Average Pooling
  - Calcula el valor promedio de la ventana
  - Matiene el contexto global de la imagen, sin enfatizar puntos fuertes, útil al analizar imágenes medicas, donde las diferencias son muy sutiles
- Min Pooling
  - Toma el valor mínimo
  - No es común utilizarlo, solo en casos especiales donde el valor mas bajo tiene importancia (detección de sombras o regiones oscuras)

Componentes

Capa de activación

- Introduce no linealidad, normalmente **ReLU**

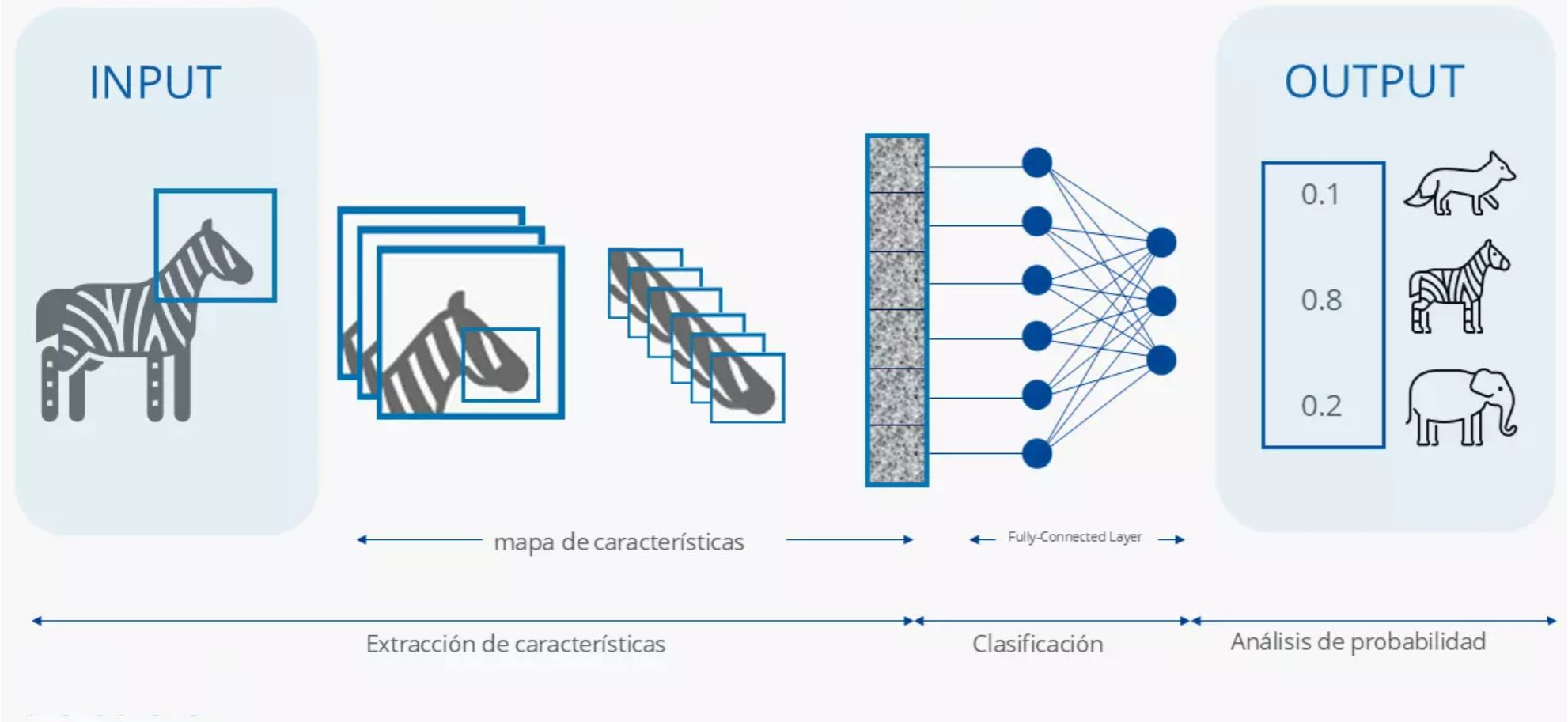
Capa completamente conectada (fully connected)

- **Conecta** las características extraídas en capas anteriores con las salidas finales
- Aquí se toma la decisión de clasificación

Capa de salida

- Utiliza funciones **Softmax** para convertir las activaciones en probabilidades de pertenencia a cada clase.

Componentes



Ventajas

- Reducción del numero de parámetros gracias al uso de filtros compartidos
- Aprendizaje automático de características
- Excelente rendimiento en tareas de clasificación, detección y segmentación de imágenes.
- Realización ante pequeñas variaciones.

Usos comunes

- Clasificación de imágenes
- Reconocimiento facial
- Detección de objetivos dinámicos/estáticos vehículos autónomos
- Diagnostico medico asistido por imágenes
- Reconocimiento de escritura o digitos

Demostración

Ejercicio

- Realizar el siguiente ejercicio para comprender cómo el número de filtros y el tipo de pooling (Max o Average) afectan: la capacidad de extracción de características, la complejidad del modelo, y la precisión en clasificación de imágenes.
  - Realizar x ejercicios con:
    - Maxpooling y pocos filtros: 16/32
    - Maxpooling con mas filtros: 64
    - MaxPolling con muchos filtros: 128
    - Averagepooling y pocos filtros: 16/32
    - Averagepooling con mas filtros: 48/64
    - Averagepooling y con muchos filtros: 128
- Registre los valores de perdida, preciscion, tiempo de entrenamiento.
- Explicar: ¿Qué tipo de pooling funcionó mejor?¿Qué efecto tuvo aumentar el número de filtros?¿Hubo riesgo de sobreajuste?